

Listas e identidade de objetos em Python

Manipulação de listas com `append()`, `insert()`, `remove()`, `pop()`, alteração de valores e `clear()`

Curso	Engenharia de Software
Disciplina	Estrutura de Dados II
Professora	Karla Roberta
Estudantes	Pietro Vianna da Rocha João Pedro Alves de Sousa
Ambiente	Google Colab - Python 3

Objetivo

Investigar, por meio de operações sucessivas em uma lista Python, como a identidade do objeto lista e a identidade dos elementos se comportam durante modificações no conteúdo.

1. Proposta e metodologia

O programa parte da lista **[10, 20, 30]** e executa uma operação por vez. Após cada etapa são exibidos o conteúdo atual da lista, seu **id()** e, para cada elemento existente, o índice, o valor e o respectivo **id()**.

O uso de **id()** permite observar a identidade dos objetos durante a execução. Os números apresentados nesta execução são apenas evidências de uma execução específica: eles podem variar quando o notebook é executado novamente ou em outro ambiente.

Etapa	Operação	Finalidade
1	Lista inicial	Registrar o estado de partida
2	append(40)	Adicionar um valor ao final
3	insert(1, 15)	Inserir um valor em posição específica
4	remove(20)	Remover pelo valor
5	pop(2)	Remover pela posição
6	lista[0] = 99	Alterar um elemento existente
7	clear()	Remover todos os elementos

2. Código Python

O código abaixo foi estruturado para que a função **mostrar_estado()** concentre a exibição das informações solicitadas, evitando repetição e facilitando a leitura.

```
lista = [10, 20, 30]

def mostrar_estado(titulo):
    print("\n" + "=" * 55)
    print(titulo)
    print("=" * 55)
    print("Lista atual:", lista)
    print("id() da lista:", id(lista))

    if len(lista) == 0:
        print("A lista está vazia.")
    else:
        for indice, valor in enumerate(lista):
            print(f"Índice: {indice} | Valor: {valor} | id(): {id(valor)}")

mostrar_estado("1. LISTA INICIAL")

lista.append(40)
mostrar_estado("2. APÓS append(40)")

lista.insert(1, 15)
mostrar_estado("3. APÓS insert(1, 15)")

lista.remove(20)
mostrar_estado("4. APÓS remove(20)")

elemento_removido = lista.pop(2)
print("Elemento removido:", elemento_removido)
print("id() removido:", id(elemento_removido))
mostrar_estado("5. APÓS pop(2)")

id_anterior = id(lista[0])
valor_anterior = lista[0]
```

```
lista[0] = 99
print("Valor anterior:", valor_anterior, "| id():", id_anterior)
print("Novo valor:", lista[0], "| id():", id(lista[0]))
mostrar_estado("6. APÓS ALTERAR lista[0] PARA 99")

lista.clear()
mostrar_estado("7. APÓS clear()")
```

3. Evidências da execução

A seguir estão os estados registrados durante uma execução do programa. O valor de **id()** da lista permaneceu constante em todas as operações realizadas sobre o mesmo objeto.

1. Lista inicial

Lista atual: [10, 20, 30] id() da lista: 140341673776064

Índice	Valor	id() do elemento
0	10	10738704
1	20	10739024
2	30	10739344

2. Após append(40)

Lista atual: [10, 20, 30, 40] id() da lista: 140341673776064

Índice	Valor	id() do elemento
0	10	10738704
1	20	10739024
2	30	10739344
3	40	10739664

3. Após insert(1, 15)

Lista atual: [10, 15, 20, 30, 40] id() da lista: 140341673776064

Índice	Valor	id() do elemento
0	10	10738704
1	15	10738864
2	20	10739024
3	30	10739344
4	40	10739664

4. Após remove(20)

Lista atual: [10, 15, 30, 40] id() da lista: 140341673776064

Índice	Valor	id() do elemento
0	10	10738704

Índice	Valor	id() do elemento
1	15	10738864
2	30	10739344
3	40	10739664

5. Após pop(2)

Lista atual: [10, 15, 40] id() da lista: 140341673776064

Elemento removido: 30 | id(): 10739344

Índice	Valor	id() do elemento
0	10	10738704
1	15	10738864
2	40	10739664

6. Após alterar lista[0] para 99

Lista atual: [99, 15, 40] id() da lista: 140341673776064

Antes: valor 10, id 10738704 | Depois: valor 99, id 10741552

Índice	Valor	id() do elemento
0	99	10741552
1	15	10738864
2	40	10739664

7. Após clear()

Lista atual: [] id() da lista: 140341673776064

A lista está vazia; portanto, não existem elementos para detalhar.

4. Análise dos resultados

a) O `id()` da lista mudou durante as operações? O que isso indica?

Não. Nas operações realizadas - `append()`, `insert()`, `remove()`, `pop()`, alteração direta de elemento e `clear()` - a lista continuou sendo o mesmo objeto. Isso indica que essas operações modificam o conteúdo da lista existente sem substituir o objeto lista por outro.

b) O que aconteceu com os elementos quando novos valores foram adicionados?

A lista passou a armazenar referências para novos objetos. Ao usar `append(40)` ou `insert(1, 15)`, novos valores passaram a fazer parte do conteúdo da lista, cada um com sua própria identidade observável por `id()`, enquanto a identidade da lista foi preservada.

c) O que acontece com a referência de um elemento quando ele é removido da lista?

A lista deixa de manter aquela referência. No caso de `pop()`, o valor removido é retornado pela operação e pode continuar acessível se for armazenado em outra variável, como ocorreu com `elemento_removido`.

d) Ao alterar um elemento da lista, o `id()` desse elemento permaneceu igual? Explique.

Não. Ao executar `lista[0] = 99`, a posição 0 deixou de referenciar o objeto anterior e passou a referenciar o objeto correspondente ao valor 99. Como inteiros são objetos imutáveis em Python, o inteiro existente não é alterado internamente; a referência armazenada naquela posição é substituída.

e) Por que dizemos que uma lista em Python é um objeto mutável?

Porque seu conteúdo pode ser modificado após a criação sem que seja necessário criar outra lista. É possível adicionar, inserir, remover, substituir e limpar elementos mantendo a mesma identidade do objeto lista.

f) Qual é a diferença entre alterar o conteúdo de uma lista e criar uma nova lista?

Alterar o conteúdo modifica o objeto existente. Já uma atribuição como `lista = [1, 2, 3]` cria uma nova lista e faz a variável passar a referenciar esse novo objeto. Nesse segundo caso, o `id()` associado à variável tende a ser diferente do anterior.

5. Conclusão

A execução confirma a principal característica das listas em Python: elas são **mutáveis**. O objeto lista mantém sua identidade mesmo quando seu conteúdo é alterado. Já os elementos armazenados podem ser incluídos, removidos ou substituídos, fazendo com que as referências internas da lista mudem ao longo da execução.

A observação de `id()` torna essa diferença visível e ajuda a distinguir claramente a identidade do contêiner da identidade dos objetos armazenados nele.